FINAL YEAR PROJECT PROPOSAL

PROJECT A.U.R.A. (Autonomous Universal Reasoning & Action Agent)

A 100% Offline, Biometrically-Gated OS Automation Agent using Hybrid Quantized LLMs, Accessibility Hooks, and Headless IoT Projection.

Core Domains: Edge AI, OS Kernel Sandboxing, Hybrid Quantized Models (1.58-bit + Low-bit), Hardware-in-the-Loop (HITL) Security, Multi-Agent Memory Systems.

1. EXECUTIVE ABSTRACT

Project A.U.R.A. is a blueprint for a next-generation local AI assistant designed to automate standard personal computers without relying on cloud APIs or compromising user data privacy. Unlike conventional cloud-based AI agents that send live screen data to third-party servers, A.U.R.A. operates 100% offline.

The system uses a hybrid local-inference strategy: a lightweight native BitNet b1.58 model (2B parameters) handles fast, low-latency routing and simple decisions in the Brain and Manager layer, while a larger 4-bit/8-bit quantized model (7-8B parameters, served via llama.cpp) handles complex multi-step reasoning inside the sub-agents. Both run entirely on-device with no GPU dependency, on a laptop with 16GB+ RAM. Combined with OS Accessibility Tree Hooks as the primary UI-navigation method, this hybrid approach delivers significantly reduced inference cost and memory footprint compared to standard floating-point models, while preserving strong reasoning capability where it is needed most.

The most distinctive feature of this project is its headless operation: even with the laptop lid closed, a desk-mounted IoT node listens to voice commands, displays output via a silent mini-projector, and uses an Optical Fingerprint Sensor to biometrically approve high-risk operating system commands — adding a hardware-enforced layer of security beyond what software-only agents provide.

2. REAL-WORLD PROBLEMS SOLVED (Project Value)

This project directly addresses four major bottlenecks in the current AI agent industry:

- **Enterprise Data Leaks (Zero-Day Privacy):** Current cloud agents (like Claude or ChatGPT) take live screenshots of the user's screen and send them to the cloud. Banks, hospitals, and privacy-conscious users cannot use them. A.U.R.A. is entirely air-gapped; not a single byte of task data leaves the local network.
- **The “Rogue AI” & Malware Threat:** If an AI agent reads malicious code from the internet (Indirect Prompt Injection), it could autonomously format a drive or delete files. A.U.R.A. mitigates this through Hardware-in-the-Loop (HITL) biometric gating, making it extremely difficult for the AI to execute critical tasks without physical human authorization.
- **High Operational Costs:** API-based agents consume thousands of tokens, costing users significant monthly fees. By using local hybrid-quantized models instead of pay-per-token cloud APIs, A.U.R.A. brings the ongoing operational cost down to $0 after the initial one-time hardware setup.
- **Headless Ambient Computing:** Users working across multiple screens, or those with physical disabilities, often cannot navigate complex multi-step UI tasks. A.U.R.A. allows them to execute background OS tasks hands-free, receiving silent visual feedback on their desk via projection without opening the laptop.

3. CORE TECHNOLOGICAL INNOVATIONS

A. Hybrid Quantized Reasoning Engine

Standard LLMs require heavy GPUs to compute floating-point matrix multiplications. A.U.R.A. uses a two-tier local model strategy: (1) Microsoft's BitNet b1.58 2B4T — the first publicly available, natively-trained 1.58-bit model — handles the Brain and Manager's fast routing and decision-making layer, where ternary {-1, 0, 1} weights replace heavy multiplication with simple addition; (2) a standard 4-bit/8-bit GGUF-quantized 7-8B model (e.g., Qwen2.5-7B or Llama-3.1-8B) via llama.cpp handles complex sub-agent reasoning that needs stronger language understanding. This hybrid design exists because no native 1.58-bit model above 2B parameters is currently publicly available — training one from scratch is far beyond the compute budget of an FYP — so the project applies BitNet's efficiency where it is proven, and conventional local quantization where more reasoning capacity is required.

B. Background Process Targeting with Vision Fallback

Traditional UI agents bring windows to the foreground and mimic mouse clicks, which interrupts the user's current work. A.U.R.A. primarily uses Windows OS Accessibility APIs (pywinauto) to target specific Process IDs (PIDs) and UI element trees — for example, locating the WhatsApp tab among 10 open browser tabs and sending a message completely in the background, without minimizing or maximizing any window.

Because third-party UI layouts (e.g., WhatsApp Web, LinkedIn) change without notice and can break fixed accessibility-tree selectors, A.U.R.A. includes a lightweight local vision fallback: when an expected UI element is not found in the accessibility tree, the system takes a background screenshot of just the relevant window and uses a small local vision model to re-locate the element visually — the same way a person re-orients themselves after an interface update — before resuming accessibility-tree-based control. This fallback activates only on failure, keeping the primary interaction path fast and lightweight.

C. Hierarchical Long-Term Memory

AI agents typically lose context once a session ends. A.U.R.A. uses ChromaDB (vector embeddings) as long-term memory to remember user habits, preferences, and file paths — for example, automatically knowing where a user's developer folders are stored, or which Chrome profile they prefer for a given task.

D. Zero-Trust Security & Sanitized Sandboxing

To protect against malicious web code, the system's web scraper strips all HTML and JavaScript, feeding only plain text to the AI inside an isolated sandbox. The AI operates in a Restricted Access Sandbox: it has permission to edit project files but is strictly blocked from modifying core OS files (e.g., System32).

E. Platform-Compliant Interaction Layer

To reduce the risk of triggering automated-behavior detection on third-party platforms (e.g., WhatsApp Web, LinkedIn), the Communication Agent enforces randomized human-like interaction timing, natural (non-linear) cursor movement paths, and per-platform daily action-rate limits. This does not remove the underlying policy considerations of automating a third-party platform — which the project treats as a documented risk and testing constraint — but it substantially reduces the chance of behavioral-pattern-based account restriction during controlled testing.

4. SYSTEM ARCHITECTURE (Software & Hardware)

Software Stack

- **Local AI Brain:** Ollama / llama.cpp — BitNet b1.58 2B4T (native 1.58-bit) for fast routing, plus a 4-bit/8-bit quantized 7-8B model (e.g., Qwen2.5-7B, Llama-3.1-8B) for complex reasoning.
- **OS Automation:** PyWinAuto, PyAutoGUI, and Windows AppContainer (kernel-level restriction), plus a lightweight local vision-fallback module for UI-change resilience.
- **Database:** ChromaDB (Vector) + SQLite (Relational State Logging), encrypted with AES-256.

Hardware IoT Node

The physical companion device is engineered around a Raspberry Pi (Zero 2W or RPi 4).

- **Why Raspberry Pi?** A standard microcontroller (like an ESP32) lacks the processing power to stream complex frame-buffers to a projector, process local wake-words seamlessly, and securely encrypt biometric hashes. The Raspberry Pi's Linux environment handles these demanding tasks efficiently.
- **AS608 Optical Fingerprint Sensor:** Acts as the physical authentication gateway.
- **INMP441 I2S Omnidirectional Mic:** Captures user voice commands while canceling background noise.
- **Mini Laser/OLED Projector:** Silently projects system prompts and feedback onto the desk or wall, eliminating the need for disruptive text-to-speech audio.

5. MARKET COMPETITORS & PRIOR ART

Competitor Product	Core Methodology	A.U.R.A.'s Innovation & Advantage	Reference
Claude Computer Use	Takes UI screenshots and sends them to cloud APIs for processing.	100% Offline: Claude Computer Use sends screenshots to the cloud, adding network latency and data-privacy exposure. A.U.R.A. runs entirely on-device using local OS Accessibility Hooks, with a vision fallback used only when needed.	Anthropic Docs
Microsoft UFO	Automates Windows UI but relies on paid GPT-4V APIs.	Zero Ongoing Cost: UFO relies on paid GPT-4V API calls. A.U.R.A. runs entirely on free, local hybrid-quantized models after the initial one-time hardware setup.	Microsoft GitHub
Open Interpreter	Terminal-based agent executing Python code locally.	Hardware Security: Open Interpreter can accidentally damage an OS with no physical safeguard. A.U.R.A. features a Fingerprint HITL lock that physically blocks high-risk actions until biometrically approved.	Open Interpreter

6. SCIENTIFIC RESEARCH FOUNDATION

This project is built upon the following peer-reviewed computer science architectures:

- **1. 1-Bit LLM Architecture —** “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits” (Microsoft Research, 2024). Provides the mathematical foundation for BitNet b1.58 2B4T, the native 1.58-bit model used in A.U.R.A.'s fast-routing layer. (arXiv:2402.17764)
- **2. OS Automation via Accessibility Hooks —** “OS-World: Benchmarking Multimodal Agents in Real Computer Environments” (NeurIPS 2024). Supports the claim that accessibility-tree parsing is faster and more reliable than pure vision-based screenshot analysis for most UI tasks — the basis for A.U.R.A.'s accessibility-first, vision-fallback design. (arXiv:2404.07972)
- **3. Agentic Memory Systems —** “MemGPT: Towards LLMs as Operating Systems” (UC Berkeley, 2023). Outlines the methodology for implementing virtual memory (vector DBs) to give AI agents long-range context and recall. (arXiv:2310.08560)

7. TECHNICAL EXECUTION SCENARIO (Real-World Pipeline)

Scenario: The user wants to back up a local database and deploy a codebase via SSH to a production server.

- Headless Voice Input: With the laptop lid closed, the user speaks to the desk device: “A.U.R.A, back up the backend project and deploy it to the production server.”
- Background Navigation: A.U.R.A. retrieves the project path from its vector memory. It uses OS accessibility hooks to silently open a hidden background terminal, compresses the folder, and initiates the deployment script without disrupting any active screens.
- Kernel Sandbox Freeze: The AI attempts to access the SSH private key (id_rsa). The OS kernel sandbox instantly flags this as a “High-Risk Restricted Action” and freezes the AI's execution.
- Silent Projection Trigger: The Raspberry Pi receives the freeze state and silently projects a warning onto the desk: [CRITICAL ACTION: SSH Key Access Requested. Verify Fingerprint.]
- Hardware Interrupt: The user places their thumb on the AS608 sensor. The Raspberry Pi validates the biometric hash and sends a secure WebSocket token to the PC.
- Final Execution: The kernel unfreezes the AI, the code is deployed, and the desk projector silently displays: [Deployment Successful — Live on Port 8080].

8. EXPECTED FINAL DELIVERABLE

For the final defense, the setup will showcase a closed laptop paired with a custom 3D-printed Raspberry Pi IoT node. The demonstration will prove that complex, enterprise-level PC workflows (file management, code deployment, targeted messaging) can be executed autonomously in the background via voice.

The highlight of the deliverable will be the Biometric Hardware-in-the-Loop system, where the system will actively refuse to execute a high-risk command (like deleting a sensitive file) until the supervisor physically provides a fingerprint scan. This project aims to demonstrate that a practical path for local Edge AI lies in combining hybrid-quantized on-device inference with biometric hardware fusion.